



ARQUITETURA HEXAGONAL

1. FUNDAMENTOS E PRINCÍPIOS

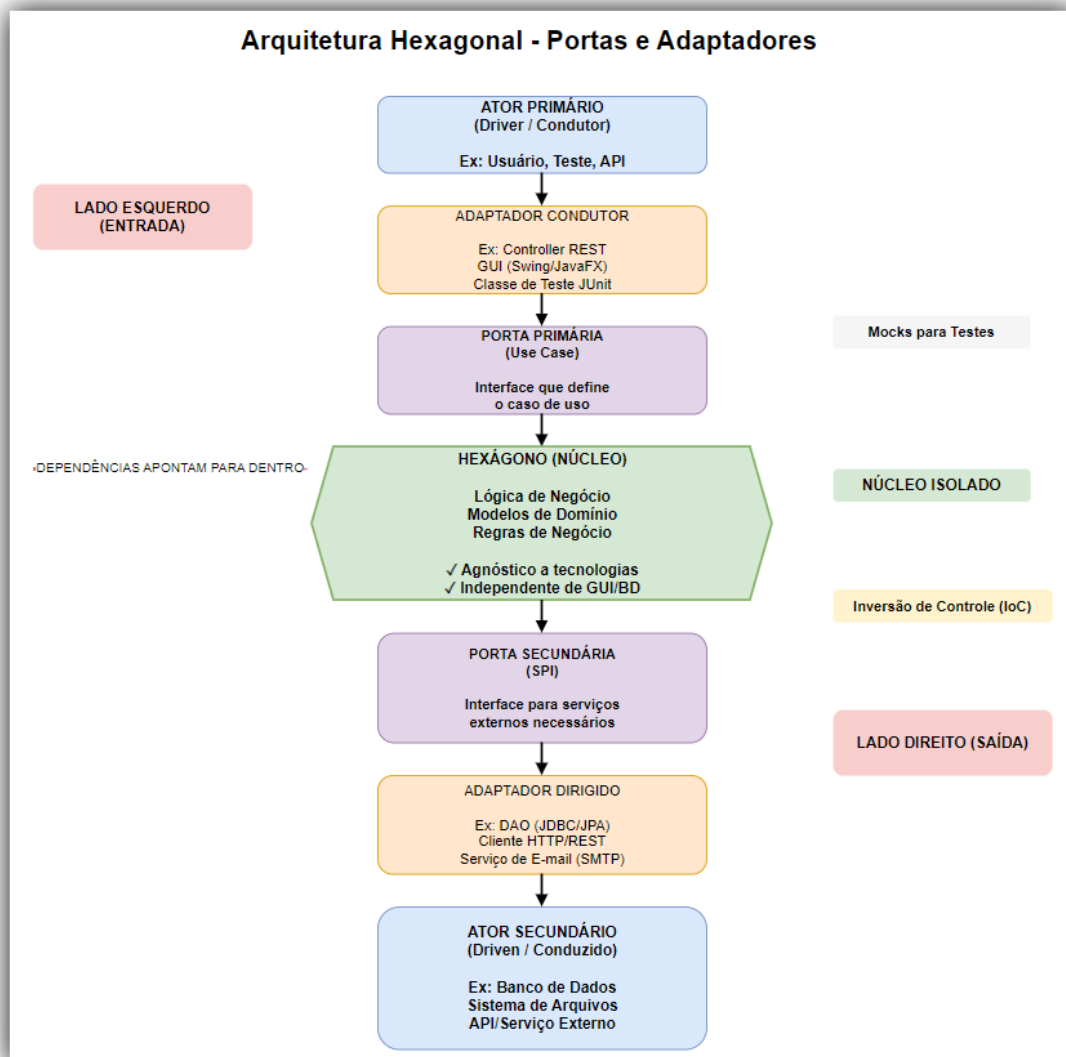
A Arquitetura Hexagonal, também conhecida como **Padrão de Portas e Adaptadores (Ports and Adapters Pattern)**, foi proposta por **Alistair Cockburn** em **2005**, um dos autores do **Manifesto Ágil**.

Seu surgimento está diretamente ligado à necessidade de alinhar a arquitetura de software com os princípios do desenvolvimento ágil, promovendo:

- **Flexibilidade** para mudanças tecnológicas;
- **Testabilidade** independente de interfaces ou infraestrutura;
- **Foco no domínio** (regras de negócio), isolando-o de detalhes técnicos.

A arquitetura permite que o sistema seja executado por diferentes tipos de atores e usuários, programas, testes automatizados sem que sua lógica central seja afetada.

2. PRINCÍPIOS E METÁFORAS





2.1. Isolamento (Separation of Concerns – SoC)

A arquitetura aplica o princípio de separação de preocupações, dividindo a aplicação em **três áreas principais**:

1. **Centro (Hexágono)** - Contém toda a lógica de negócio, modelos de domínio e casos de uso. É completamente **agnóstico a tecnologias**, frameworks ou infraestrutura externa. Pode depender apenas de serviços gerais como logging e injeção de dependência.
2. **Lado Superior Esquerdo** - Responsável pela interação com atores externos que **iniciam** a comunicação. Contém código específico de tecnologia, como interfaces gráficas, endpoints REST ou filas de mensagens.
3. **Lado Inferior Direito** - Responsável por fornecer serviços de infraestrutura que o sistema necessita, como banco de dados, envio de e-mails, sistemas de arquivos e chamadas HTTP.

Legenda: O hexágono representa o núcleo da aplicação. As portas estão na borda. Os adaptadores convertem chamadas entre tecnologias específicas e o núcleo.

2.2. Atores

São entidades do mundo real com as quais a aplicação interage: humanos, outros sistemas, dispositivos.

- **Ator Primário (Condutor / Driver)** - Inicia a interação. Ex.: usuário, teste automatizado, sistema externo que chama uma API.
- **Ator Secundário (Conduzido / Driven)** - É acionado pelo sistema. Ex.: banco de dados, serviço de e-mail, sistema de arquivos.

Regra de descoberta: Pergunte-se quem inicia a comunicação. Se for o ator → primário. Se for o sistema → secundário.

2.3. Dependências

A arquitetura define a seguinte regra: **as dependências devem apontar de fora para dentro**.

- O hexágono **não depende** de ninguém.
- Atores e adaptadores **dependem do hexágono**.

Em tempo de execução, essa regra é garantida por meio de **Inversão de Controle (IoC)**.

2.4. Portas (Ports)

Portas são **interfaces (contratos)** que definem a comunicação entre o hexágono e o mundo externo.

- **Porta Primária (Driver)** - Define os casos de uso (Use Cases)
Ex.: *AdicionarItemCarrinho*, *RealizarTransferencia*.
- **Porta Secundária (Driven)** - Define serviços que o hexágono necessita.
Ex.: *RepositorioCliente*, *ServicoEnvioEmail*.

2.5. Adaptadores (Adapters)

Adaptadores são implementações concretas que **traduzem** chamadas entre uma tecnologia específica e uma porta.

- **Adaptador Condutor:** Ex.: Controlador REST, Interface Gráfica (Swing/JavaFX), Classe de Teste JUnit.
- **Adaptador Dirigido:** Ex.: DAO com JDBC/JPA, Cliente HTTP, Serviço SMTP.

Regra: Para cada porta, deve haver **pelo menos dois adaptadores**:

1. Um para **testes** (TDD);
2. Um para a **tecnologia real** de produção.



2.6. Adaptadores Simulados (Mocks)

Para garantir o isolamento total, a arquitetura recomenda o uso de **mocks** que emulam dispositivos externos:

- Banco de dados em memória (HashMap);
- Envio de e-mail via `System.out.println()`;
- Cliente HTTP simulado.

Isso permite que o hexágono seja desenvolvido, testado e homologado **sem dependências externas reais**.

2.7. Dependências Configuráveis

A configuração dinâmica de adaptadores é feita por meio de **IoC** (ex.: Spring, CDI, Pico Container), permitindo que o sistema utilize diferentes adaptadores conforme o ambiente (desenvolvimento, teste, produção).

3. EXEMPLO REAL: GERENCIADOR DE TAREFAS

Sistema onde funcionários atribuem tarefas entre si. Quando uma tarefa é atribuída, um e-mail é enviado ao responsável.

Estrutura:

- **Porta Primária:** *AtribuirTarefaUseCase*
- **Porta Secundária:** *NotificadorEmail*
- **Adaptador Primário:** Controlador Web (Spring MVC)
- **Adaptador Secundário:** Serviço SMTP (*JavaMail*) ou Mock para testes

4. QUANDO USAR OU NÃO USAR?

Recomendado para:

- Projetos de **médio a grande porte** com **longo ciclo de vida**;
- Sistemas que precisam ser **altamente testáveis**;
- Projetos que exigem **flexibilidade** para mudar tecnologias de Frontend ou infraestrutura;
- Equipes que adotam **TDD** como prática.

Não recomendado para:

- Projetos **pequenos, temporários ou legados**;
- Sistemas com regras de negócio **fortemente acopladas a bancos de dados proprietários**;
- Casos onde não há previsão de mudanças tecnológicas.

5. CONCLUSÃO

A Arquitetura Hexagonal não é uma "bala de prata", mas se destaca como uma excelente opção para sistemas complexos e em evolução, o custo inicial de abstração é compensado pela **facilidade de manutenção, testabilidade e adaptabilidade** a mudanças.

Ela se alinha com outras abordagens modernas como **DDD (Domain-Driven Design)**, **Onion Architecture** e **Clean Architecture**.

6. REFERÊNCIAS COMPLEMENTARES

- Documento original:
https://web.archive.org/web/20080914152611/http://alistair.cockburn.us/index.php/Hexagonal_architecture
- Tradução e melhorias:
<https://fernandofranzini.wordpress.com/2019/04/09/arquitetura-hexagonal/>

EducaCiência FastCode para a comunidade